Lesson Plan: Module 5

System Validation, Stress Testing, Benchmarking, and Final Documentation

---

An Educational Framework for AI-Driven Signal Processing

Student:	Jude Eschete
Project Advisor:	Dr. Bernard Yett
Program:	M.S. Computer Engineering
Institution:	Stevens Institute of Technology
Date Range:	April 26 – May 9, 2026

This document presents the lesson plan for the fifth and final module of the capstone project. Students subject the complete end-to-end system—beacon array, radio frequency (RF) link, field-programmable gate array (FPGA) processing pipeline, and machine learning (ML) classification—to sustained stress tests, benchmark total system response time from transmission to classification output, analyze failure modes under adversarial and edge-case conditions, finalize all curriculum documentation and lab guides, and prepare the final project report and presentation.

JEschete@stevens.edu

Contents

1 Module Overview **1**
- 1.1 Context Within the Project . . . 1
- 1.2 Prerequisites . . . 2

2 Weekly Schedule **3**

3 Week 1: System Stress Testing **4**
- 3.1 Objectives . . . 4
- 3.2 Stress Test Philosophy . . . 4
- 3.3 Stress Test Plan . . . 4
  - 3.3.1 S1: Beacon Density Stress Test . . . 4
  - 3.3.2 S2: Rapid Parameter Change Test . . . 6
  - 3.3.3 S3: Low-signal-to-noise ratio (SNR) Stress Test . . . 7
  - 3.3.4 S4: Extended-Duration Stability Test . . . 8

4 Week 2: End-to-End Benchmarking **10**
- 4.1 Objectives . . . 10
- 4.2 Response Time Decomposition . . . 10
  - 4.2.1 general-purpose input/output (GPIO) Toggle Measurement Method . . . 10
- 4.3 Benchmarking Matrix . . . 11
- 4.4 Baseline Comparison . . . 12
- 4.5 System Specification Table . . . 12

5 Week 3: Failure Analysis and Curriculum Packaging **14**
- 5.1 Objectives . . . 14
- 5.2 Failure Mode Catalog . . . 14
- 5.3 Curriculum Package . . . 15
  - 5.3.1 Package Contents . . . 15
  - 5.3.2 Instructor's Guide . . . 15

6 Week 4: Final Report and Presentation **17**
- 6.1 Objectives . . . 17
- 6.2 Final Report Structure . . . 17
- 6.3 Presentation Requirements . . . 18
  - 6.3.1 Suggested Slide Outline . . . 18

7 Complete System Architecture: Final Validated Pipeline **20**

8 Hardware and Software Reference **21**

9 Assessment Rubric **21**

10 Recommended Resources **22**
10.1 System Testing and Validation 22
10.2 Technical Writing and Presentation 22
10.3 Cross-Domain System Design 22
10.4 Project-Specific References 22

11 Framework Retrospective **23**
11.1 Cross-Module Dependency Summary 23
11.2 The Framework's Central Lesson 24

1 Module Overview

Module 5 Learning Objectives

Upon completion of this module, the student (or future learner following this curriculum) will be able to:

LO-1 Design and execute a comprehensive stress test plan that pushes the end-to-end system beyond its nominal operating envelope, using multiple concurrent beacons at maximum transmission rates, rapid parameter changes, and extended-duration runs.

LO-2 Benchmark the total system response time from beacon transmission to classification output, decomposing it into per-stage latency contributions and identifying the dominant bottleneck at each operating point.

LO-3 Characterize classifier accuracy degradation under stress conditions (high beacon density, low SNR, edge-of-range deployment) and compare against the Module 4 performance-vs.-condition baselines.

LO-4 Identify, categorize, and document system failure modes—pipeline stalls, first in, first out (FIFO) overflows, classification accuracy collapse, and RF channel saturation—with root-cause analysis tracing each failure to a specific design decision in Modules 1–4.

LO-5 Synthesize results from all five modules into a coherent final project report that demonstrates the cross-domain reasoning central to the framework's pedagogical goals.

LO-6 Prepare and deliver a technical presentation that communicates the system architecture, experimental results, and lessons learned to an audience of faculty and peers.

LO-7 Package the complete curriculum—lesson plans, source code, datasets, trained models, and lab guides—into a self-contained deliverable that a future instructor or student could use independently.

1.1 Context Within the Project

Module 5 is the capstone of the capstone. Modules 1–4 each built, validated, and characterized a subsystem or processing stage; this module evaluates the *integrated whole* under conditions that no individual module tested. The stress tests in Weeks 1–2 are designed to find the system's breaking points: the beacon density at which the FPGA pipeline can no longer keep up, the SNR threshold below which classification accuracy becomes unreliable, the runtime duration beyond which cumulative errors corrupt the dataset. These breaking points are not failure—they are the system's specification, derived empirically rather than assumed.

The final documentation and presentation in Weeks 3–4 are not afterthoughts. The framework's educational value is only fully realized when a student can articulate *why* the system behaves as it does: why a firmware timing decision in Module 1 limits the maximum beacon rate in Module 5, why the cyclic redundancy check (CRC) polynomial choice in Module 2 determines the undetected error rate under channel stress, why the feature vector design in Module 2 constrains classification accuracy in Module 4. These cross-module cause-and-effect chains are the subject of the final report and the core of the presentation.

1.2 Prerequisites

Students beginning this module should have:

- Completed Modules 1–4 with:
  - A working multi-beacon RF subsystem ($\geq$ 2 beacons + 1 receiver).
  - A deployed FPGA pipeline (Module 2 bitstream running on the Nexys A7-100T).
  - A characterized end-to-end data path with measured packet error rate (PER), latency, and throughput (Module 3).
  - Trained classifiers for SF detection, modulation classification, and emitter ID (Module 4) with serialized model and scaler files.
  - A real-time inference script validated for live classification (Module 4 Week 4).
- All hardware assembled, wired, and verified (STM32 beacons, STM32 receiver, Nexys A7, host PC).
- The Module 3 labeled dataset and Module 4 model comparison results available for reference.
- Familiarity with technical report writing and presentation preparation.

2 Weekly Schedule

The module spans two weeks (four working sessions). The first week focuses on stress testing and benchmarking; the second week focuses on failure analysis, documentation, and presentation.

Table 1: Module 5 weekly breakdown.

Week	Focus Area	Key Activities
1	System Stress Testing	Design stress test plan; execute multi-beacon density tests; rapid parameter change tests; extended-duration stability run; record all metrics (PER, latency, FIFO utilization, classification accuracy).
2	End-to-End Benchmarking	Measure total response time (TX to classification) with per-stage decomposition; benchmark at multiple operating points (SF, beacon count, distance); compare stressed performance against Module 3/4 baselines; identify dominant bottleneck per operating point.
3	Failure Analysis & Curriculum Packaging	Categorize and document all observed failure modes with root-cause tracing to originating module; finalize lab guides for each module; package source code, datasets, and models; write the curriculum instructor's guide.
4	Final Report & Presentation	Write the final project report synthesizing all five modules; prepare presentation slides; rehearse and deliver the final presentation; submit complete curriculum package.

3 Week 1: System Stress Testing

3.1 Objectives

- Design a structured stress test plan that systematically pushes each system dimension beyond its nominal operating envelope.
- Execute beacon density stress tests with increasing numbers of concurrent transmitters.
- Execute rapid parameter change tests to evaluate classifier tracking ability.
- Execute an extended-duration stability test ($\geq$30 minutes) to detect cumulative drift, counter overflow, or memory-related issues.
- Record comprehensive metrics at each stress level for comparison against Module 3 baselines.

3.2 Stress Test Philosophy

Design Principle: Finding the Specification—Not Proving Compliance

A stress test that merely confirms the system works under nominal conditions has limited educational value. The purpose of Module 5 stress testing is to find where the system *stops* working—and to understand *why*. Each failure point traces back to a specific design decision in an earlier module:

- The maximum beacon rate is constrained by the universal asynchronous receiver/transmitter (UART) bandwidth chosen in Module 1 (115200 baud $\rightarrow$ 268 fps theoretical max) and the FIFO depth chosen in Module 2 (64 entries).
- The minimum classifiable SNR is constrained by the feature vector design in Module 2 and the classifier training data distribution in Module 3.
- The maximum sustained runtime is constrained by counter widths (32-bit sequence numbers, 16-bit observation counts) and ring buffer sizing decisions in Module 1.

The stress tests *derive* the system specification from empirical evidence, rather than testing against an a priori specification. This is the engineering approach: characterize, don't assume.

3.3 Stress Test Plan

The stress test plan covers four dimensions, each targeting a different system limitation:

3.3.1 S1: Beacon Density Stress Test

This test determines the maximum number of concurrent beacons the pipeline can handle without unacceptable PER degradation or FIFO overflow.

For each configuration, record:

1. Total packets transmitted (sum of all beacon sequence numbers).
2. Total valid feature vectors produced (FPGA observation counts).

Table 2: Stress test dimensions and configurations.

ID	Dimension	Configurations	Duration
S1	Beacon Density	1, 2, 3 beacons; fixed 1 s interval; then ALOHA [0.5, 1.5] s	5 min each
S2	Rapid Parameter Change	Beacon SF changes every 30 s (SF7→SF10→SF12→SF7); 2 beacons	10 min
S3	Low-SNR Operation	TX power swept from +20 to +2 dBm; distance increased to 8 m; 1 beacon	3 min/level
S4	Extended Duration	2 beacons, nominal parameters (SF7, +14 dBm, 2 s interval)	30 min

Table 3: S1: Beacon density test configurations.

Config	Beacons	Mode	Interval	Target Observation
S1a	1	Fixed	1 s	Baseline: confirm near-zero PER and stable FIFO.
S1b	2	Fixed	1 s each	Staggered arrival; FIFO should handle 2 fps easily.
S1c	3	Fixed	1 s each	3 fps; approaching the rate where inter-frame gaps shrink.
S1d	2	ALOHA	[0.5, 1.5] s	Random collisions; burst arrivals stress the FIFO.
S1e	3	ALOHA	[0.5, 1.5] s	Maximum contention; expect measurable PER increase and FIFO peaks.

3. PER and its decomposition (RF loss, framing errors, CRC rejects, FIFO overflow).
4. FIFO peak utilization (maximum `fifo_count` observed via integrated logic analyzer (ILA) or LED mapping).
5. Classification accuracy (run the inference script simultaneously and compare predictions against known beacon configurations).

Exercise: Density Stress Test

1. Execute configurations S1a through S1e sequentially, allowing 1 minute of stabilization between configuration changes.
2. For each configuration, record all five metric categories above.
3. Plot PER vs. beacon count, separating fixed-interval and ALOHA modes.
4. Plot FIFO peak utilization vs. configuration. If any configuration exceeds 75% FIFO utilization, flag it as a capacity risk.
5. Plot classification accuracy vs. beacon count. Identify the beacon count at which accuracy drops below 90% for the SF detection task.
6. For the highest-density ALOHA configuration (S1e), capture an ILA trace showing a burst of back-to-back frames and verify the parser's resynchronization behavior.

3.3.2 S2: Rapid Parameter Change Test

This test evaluates whether the classifier can track a beacon that changes its spreading factor dynamically. In operational scenarios (e.g., LoRa adaptive data rate), emitters may change SF in response to link quality. The classifier must update its predictions accordingly rather than latching onto the initial classification.

Test Protocol: S2

1. Configure two beacons at SF7, +14 dBm, fixed 2-second interval.
2. Start the real-time inference script. Verify both beacons classified as SF7.
3. At $t = 30$ s, reflash or reconfigure Beacon 1 to SF10. Observe:
   - How many packets elapse before the classifier reports SF10 for Beacon 1?
   - Does Beacon 2's classification remain stable at SF7?
   - Does the inter-arrival time feature reflect the longer SF10 on-air time?
4. At $t = 60$ s, change Beacon 1 to SF12. Record the same observations.
5. At $t = 90$ s, return Beacon 1 to SF7. Verify the classifier tracks back.
6. Record the full 10-minute inference log with timestamps and predicted classes.

Transition Behavior Expectations

The Random Forest and MLP classifiers from Module 4 classify each feature vector independently—they have no memory of previous classifications. Therefore, each individual prediction should be correct (or incorrect) based solely on the features of that single packet. There is no "tracking delay" in the model itself. However, the FPGA's computed features—inter-arrival time and received signal strength indicator (RSSI) delta—are computed relative to the *previous* observation from the same Beacon ID. During a parameter transition, the first packet at the new SF may have an inter-arrival time that reflects the old SF's on-air time (the last packet at the old SF). This transient feature distortion may cause one or two misclassifications at the transition boundary. Document this effect and explain it in terms of the FPGA feature computation logic from Module 2.

3.3.3 S3: Low-SNR Stress Test

This test maps the classifier's accuracy degradation curve as signal quality decreases, extending the Module 4 performance-vs.-SNR analysis to more extreme conditions.

Table 4: S3: Low-SNR operating conditions.

Config	TX Power	Distance	Expected SNR	Purpose
S3a	+20 dBm	3 m	High (> 10 dB)	Comfortable baseline
S3b	+14 dBm	5 m	Moderate (5–10 dB)	Nominal operating point
S3c	+8 dBm	5 m	Low (0–5 dB)	Beginning of degradation
S3d	+2 dBm	8 m	Very low (< 0 dB)	Near receiver sensitivity limit
S3e	+2 dBm	8 m + obstruction	Minimal	Extreme: partial blockage

For configuration S3e, place a physical obstruction (e.g., a metal filing cabinet, a stack of textbooks) between the beacon and receiver. This introduces multipath fading and signal attenuation beyond the free-space path loss model, creating conditions that neither the simulator nor the clean benchtop dataset captured. The purpose is to observe how the classifier responds to out-of-distribution data—RSSI and SNR values that fall outside the training data range.

Expected Results: Out-of-Distribution Behavior

At S3d and S3e, two outcomes are possible:

- **Graceful degradation:** The classifier's accuracy decreases smoothly, with misclassifications favoring adjacent spreading factors (e.g., predicting SF8 instead of SF7). This indicates the model learned robust feature relationships.
- **Catastrophic degradation:** The classifier's accuracy drops suddenly to near-random, with predictions scattered across all classes. This indicates the model memorized training-data-specific feature ranges rather than learning generalizable relationships.

Either outcome is a valid experimental result. The educational value is in diagnosing *which* outcome occurs and tracing it back to the training data distribution (Module 3), the feature engineering decisions (Module 4), and ultimately the physical system design (Module 1).

3.3.4 S4: Extended-Duration Stability Test

This test runs the system continuously for 30 minutes under nominal conditions to detect time-dependent failure modes that do not appear in shorter runs.

What to Watch For During S4

- **Counter overflow:** The 16-bit observation count (Module 2) overflows at 65,535. At 0.5 fps per beacon, this takes ∼36 hours—well beyond the 30-minute test. However, verify that the counter approaches its expected value linearly (no unexpected resets).
- **Sequence number gaps:** Compare each beacon's final sequence number against its observation count. A growing gap indicates intermittent packet loss. Plot the cumulative gap over time; a constant gap suggests a one-time desynchronization, while a growing gap suggests an ongoing issue (thermal drift in the STM32 clock, UART baud rate drift).
- **FIFO creep:** Monitor FIFO utilization over the full 30 minutes. If the baseline utilization slowly increases, the pipeline has a throughput deficit that manifests only over long runs (e.g., the parser occasionally stalls for extra cycles, and the FIFO absorbs the backlog but never fully drains).
- **Classification stability:** With fixed beacon parameters, the classifier should produce the same prediction for every packet from a given beacon. Any drift in predictions over time (e.g., increasing misclassification rate in the last 10 minutes) may indicate feature drift (gradual RSSI change from thermal effects in the RF module) or a subtle bug in the inference script (e.g., memory leak in the Python serial reader).
- **Host-side resource usage:** Monitor the Python inference script's memory consumption and CPU usage. A linear memory increase indicates a leak (common: unbounded list appends for logging). Verify that memory usage plateaus after the first minute.

Week 1 Deliverable

Stress Test Report — A document containing:

1. Stress test plan with rationale for each dimension and configuration.
2. S1 results: PER, FIFO utilization, and classification accuracy tables and plots for all five density configurations. Identification of the maximum sustainable beacon count.
3. S2 results: Full inference log with timestamps showing classifier response to SF transitions. Analysis of transient misclassifications with root-cause explanation.
4. S3 results: Classification accuracy vs. SNR/distance curve, extended to low-SNR conditions. Characterization of degradation mode (graceful vs. catastrophic).
5. S4 results: Time-series plots of observation count, FIFO utilization, and classification accuracy over 30 minutes. Assessment of long-term stability.

4 Week 2: End-to-End Benchmarking

4.1 Objectives

- Measure the total system response time from beacon transmission to classification output with per-stage decomposition.
- Benchmark response time across multiple operating points (spreading factor, beacon count, distance).
- Compare stressed-system performance against Module 3 and Module 4 baselines.
- Identify the dominant latency bottleneck at each operating point.
- Produce a comprehensive system specification table summarizing the validated operating envelope.

4.2 Response Time Decomposition

The total system response time is the interval from the moment a beacon begins transmitting a packet to the moment the host-side inference script outputs a classification result. This end-to-end latency decomposes into five stages:

$$T_{\text{total}} = T_{\text{RF}} + T_{\text{RX_FW}} + T_{\text{UART}_{\text{in}}} + T_{\text{FPGA}} + T_{\text{UART}_{\text{out}}} + T_{\text{infer}} \tag{1}$$

Table 5: Response time decomposition with measurement method.

Stage	Symbol	Measurement Method	Description
RF on-air time	T_{RF}	Calculated from SF/BW	Time for the LoRa packet to be transmitted over the air.
Receiver firmware	$T_{\text{RX_FW}}$	GPIO toggle	ISR processing + frame construction in the STM32.
UART to FPGA	$T_{\text{UART}_{\text{in}}}$	Calculated	43 bytes × 10 bits ÷ 115200 baud = 3.73 ms.
FPGA pipeline	T_{FPGA}	ILA measurement	UART RX complete to `frame_valid` assertion.
UART to host	$T_{\text{UART}_{\text{out}}}$	Calculated	35 bytes × 10 bits ÷ 115200 baud = 3.04 ms.
Host inference	T_{infer}	`time.perf_counter()`	Feature parsing + scaling + model predict.

4.2.1 GPIO Toggle Measurement Method

For precise end-to-end measurement, instrument the system with GPIO signals:

1. **Beacon side:** Assert a GPIO pin (e.g., PA5 on the Nucleo, which drives the on-board LED) at the moment `HAL_SPI_Transmit()` begins the TX command to the SX1276.
2. **Host side:** Assert a GPIO pin (via a USB-GPIO adapter or Arduino) at the moment the inference script prints the classification result.
3. **Measurement:** Capture both GPIO signals on an oscilloscope or logic analyzer. The time between the rising edges is T_{total}.

If a USB-GPIO adapter is not available for the host side, an alternative is to use the FPGA's `frame_valid` signal (directly measurable via an output pin or ILA) as the endpoint, which gives $T_{\text{total}} - T_{\text{UART}_{\text{out}}} - T_{\text{infer}}$. The host-side contributions can be added from the Module 4 inference latency measurements.

4.3 Benchmarking Matrix

Table 6: Benchmarking operating points.

ID	SF	Beacons	Distance	Power	Key Measurement
B1	SF7	1	3 m	+14 dBm	Minimum latency operating point
B2	SF10	1	3 m	+14 dBm	Mid-range latency
B3	SF12	1	3 m	+14 dBm	Maximum latency (longest on-air time)
B4	SF7	2	3 m	+14 dBm	Multi-beacon latency impact
B5	SF7	3	3 m	+14 dBm	High-density latency impact
B6	SF7	1	8 m	+2 dBm	Low-SNR impact on all stages

For each operating point, collect 50 response time measurements and report:

- Mean, standard deviation, minimum, maximum, and 99th percentile of T_{total}.
- Per-stage decomposition (Table 5).
- Identification of the dominant stage (the stage contributing >50% of T_{total}).

Expected Results: Latency Dominance Shift

At SF7 (operating points B1, B4, B5), the RF on-air time is ∼50 ms and the UART transfers contribute ∼7 ms. The dominant latency component is the RF on-air time (∼75% of total). At SF12 (operating point B3), the on-air time grows to ∼1.5 s, making the RF stage even more dominant (>95% of total). In both cases, the FPGA processing time (<0.01 ms) and host inference time (<5 ms) are negligible.
This result has an important architectural implication: moving inference from the host PC to the FPGA (on-chip neural network) would not materially reduce T_{total} because the bottleneck is the RF link, not the processing. Students should articulate this trade-off: on-FPGA inference would eliminate the $T_{\text{UART}_{\text{out}}}$ and T_{infer} stages (∼8 ms total), which is a <15% improvement at SF7 and <1% at SF12. The investment is only justified if the UART-to-host link is eliminated entirely (e.g., for a standalone embedded system).

4.4 Baseline Comparison

Compare each benchmark result against the corresponding Module 3 and Module 4 measurements:

Table 7: Baseline comparison template.

Metric	Module 3	Module 4	Module 5	Change
PER (2 beacons, SF7)				
End-to-end latency (SF7, 3 m)		—		
SF classification accuracy	—			
Mod classification accuracy	—			
Emitter ID accuracy	—			
FIFO peak utilization (2 beacons)		—		

Regression Detection

Any significant performance degradation between Module 3/4 measurements and Module 5 measurements at the *same* operating point requires investigation. Possible causes include:

- **Hardware degradation:** Antenna loosened, USB cable partially disconnected, breadboard contact oxidation. Re-run the Module 3 pre-integration checklist (Table 2 in Module 3).
- **Firmware change:** Beacon or receiver firmware was modified during Module 4 development (e.g., baud rate changed for debugging and not reverted).
- **Bitstream change:** FPGA bitstream was regenerated with ILA probes added or removed, altering timing.
- **Environmental change:** The RF environment changed (new equipment in the lab emitting 915 MHz interference, furniture rearranged affecting multipath).

Document any regressions and their root causes in the benchmark report.

4.5 System Specification Table

The culminating output of the benchmarking effort is a formal system specification table that summarizes the validated operating envelope:

Table 8: System specification table (to be completed from measured data).

Parameter	Value	Source
Maximum concurrent beacons (PER $< 5\%$)	_____	S1
Maximum frame rate (FIFO $< 75\%$)	_____ fps	S1
Minimum classifiable SNR (accuracy $> 80\%$)	_____ dB	S3
Maximum classifiable distance (accuracy $> 80\%$)	_____ m	S3
End-to-end latency at SF7 (mean $\pm$ std)	_____ ms	B1
End-to-end latency at SF12 (mean $\pm$ std)	_____ ms	B3
SF classification accuracy (nominal)	_____%	B1
Modulation classification accuracy (nominal)	_____%	B1
Emitter ID accuracy (nominal)	_____%	B1
Maximum stable run time (tested)	_____ min	S4
FPGA resource utilization (LUT / FF / BRAM)	_____	Module 2
Host inference latency (99th percentile)	_____ ms	Module 4

Week 2 Deliverable

End-to-End Benchmarking Report — A document containing:

1. Per-stage response time decomposition for all six operating points (Table 6).
2. Response time histograms for B1 (minimum latency) and B3 (maximum latency) with mean, std, and 99th percentile annotations.
3. Bottleneck identification: a bar chart showing per-stage contribution to total latency at each operating point.
4. Baseline comparison table (Table 7) with regression analysis for any degraded metrics.
5. Completed system specification table (Table 8) with all values filled from measured data.
6. Discussion: architectural implications of the latency decomposition, including whether on-FPGA inference would be beneficial.

5 Week 3: Failure Analysis and Curriculum Packaging

5.1 Objectives

- Categorize all failure modes observed during Module 5 stress testing and Modules 1–4 development.
- Trace each failure to its originating design decision and identify which module(s) contain the root cause.
- Produce a failure mode catalog that serves as both a debugging reference and a pedagogical tool.
- Finalize all lab guides, source code, and documentation for the complete curriculum package.
- Write the curriculum instructor's guide with setup instructions, common issues, and adaptation guidance.

5.2 Failure Mode Catalog

The failure mode catalog is a structured record of every failure or degradation mode observed across the entire project. Each entry identifies what failed, how it manifested, which module's design decision caused it, and how it was (or could be) resolved.

Table 9: Failure mode catalog template (to be populated from observations).

ID	Failure Mode	Observed In	Root Module	Root Cause
F1	FIFO overflow	S1e	M2	FIFO depth (64) insufficient for burst arrivals
F2	CRC reject on valid frame	M3 Wk1	M1/M2	CRC polynomial mismatch between FW and RTL
F3	Classification collapse at low SNR	S3d	M3/M4	Training data did not cover low-SNR regime
F4	Transient misclass at SF transition	S2	M2	IAT feature uses previous-SF timing on first new-SF packet
F5	...	...	...	...

Design Principle: Failures as Cross-Domain Dependencies

The failure mode catalog is the framework's most explicit statement of its pedagogical thesis. Every row in Table 9 demonstrates a cross-domain dependency: a design decision made in one module creates a failure mode that manifests in a different module. The "Root Module" column often differs from the "Observed In" column, which is the defining characteristic of cross-domain integration issues.
Examples of cross-domain failure chains:

- **F1:** A FIFO depth decision in Module 2 (digital design domain) determines the maximum beacon density in Module 5 (system validation domain), which was driven by the UART baud rate choice in Module 1 (embedded firmware domain).
- **F3:** The training data distribution from Module 3 (data engineering domain) limits the classifier's generalization in Module 4 (ML domain), which only becomes apparent under stress in Module 5 (system validation domain), and traces back to the link budget in Module 1 (RF domain).

Students should aim for at least 8–10 catalog entries. The richness of this catalog is a direct measure of how deeply the student engaged with the cross-domain integration challenges.

5.3 Curriculum Package

The complete curriculum package must be self-contained: a future instructor or student should be able to replicate the entire project using only the materials in the package, without referring to external documentation or requesting clarification from the original developer.

5.3.1 Package Contents

5.3.2 Instructor's Guide

The instructor's guide is a new document (not a lesson plan) that provides the meta-level information needed to deploy the curriculum:

1. **Hardware Setup:** Step-by-step instructions for assembling the benchtop platform from the BOM, including photos of the wiring, antenna placement, and board connections.
2. **Software Setup:** Installation instructions for STM32CubeIDE, Vivado (WebPACK), Python environments, and all library dependencies. Specify exact version numbers that were tested.
3. **Common Issues:** A troubleshooting guide organized by symptom (e.g., "FPGA does not receive any UART data" → check ground jumper, verify PMOD pin, confirm baud rate).
4. **Adaptation Guidance:** How to modify the curriculum for different hardware (e.g., substituting a different FPGA board, using a different ISM band), different class sizes (scaling from single-student capstone to a multi-student lab), and different time constraints (condensing from 5 modules to 3).
5. **Assessment Guidance:** Suggested rubric modifications for different academic levels (senior undergraduate vs. graduate), with notes on which objectives are essential and which are stretch goals.

Table 10: Curriculum package contents checklist.

#	Item	Description	Complete?
1	Lesson Plans (Modules 1–5)	This document set; PDF and LaTeX source.	☐
2	Curriculum Sequence	Module timeline and dependency overview.	☐
3	Beacon Firmware	STM32CubeIDE project; SPI driver, packet builder, scheduler.	☐
4	Receiver Firmware	STM32CubeIDE project; ISR, ring buffer, UART forwarder.	☐
5	FPGA RTL	Vivado project; all VHDL sources, constraint file, testbenches.	☐
6	Host Collection Script	Python; serial receiver, CSV writer, campaign config.	☐
7	ML Pipeline Code	Python; `preprocess.py`, training scripts, inference script.	☐
8	Trained Models	`.pkl` / `.pt` files for RF, SVM, MLP, CNN-1D classifiers.	☐
9	Labeled Dataset	Module 3 merged CSV with train/val/test split.	☐
10	Engagement Simulator	Python simulation code and sample output.	☐
11	Hardware BOM	Bill of materials with part numbers, vendors, and costs.	☐
12	Instructor's Guide	Setup instructions, common issues, adaptation guidance.	☐

Week 3 Deliverable

Failure Analysis and Curriculum Package — Consisting of:

1. Failure mode catalog with ≥8 entries, each with root-cause analysis tracing to the originating module.
2. Cross-domain dependency diagram: a visual map showing which modules contribute to which failure modes (a directed graph from design decisions to observed failures).
3. Complete curriculum package (Table 10) with all 12 items verified present and functional.
4. Instructor's guide covering hardware setup, software setup, common issues, adaptation guidance, and assessment guidance.

6 Week 4: Final Report and Presentation

6.1 Objectives

- Write a comprehensive final project report that synthesizes results from all five modules.
- Prepare a technical presentation for faculty and peers.
- Deliver the presentation and respond to questions.
- Submit the complete curriculum package.

6.2 Final Report Structure

The final report is the definitive document of the project. It should be written for an audience that has not read the individual module deliverables and should be self-contained.

Table 11: Final report structure and content requirements.

Section	Title	Pages	Content
1	Abstract	0.5	Project summary, key results, primary conclusions.
2	Introduction	1–2	Problem statement, motivation, pedagogical goals, project scope.
3	System Architecture	2–3	Complete system diagram (beacon array → FPGA → ML); hardware and software components; inter-module data flow.
4	Module Summaries	4–5	One subsection per module: objectives, design decisions, key results. Not a rewrite—a synthesis highlighting decisions that affected downstream modules.
5	Experimental Results	3–4	Stress test results, benchmarking data, classification performance, system specification table. Focus on cross-module findings.
6	Discussion	2–3	Lessons learned; cross-domain dependency analysis; what would change if redesigned; comparison of sim vs. hardware results.
7	Conclusion	0.5–1	Summary of achievements, limitations, and future work.
	References	1–2	IEEE format; all datasheets, textbooks, and papers cited.
	Appendices	—	BOM, pin assignments, code listings (if not in the curriculum package).
	Total	**15–20**	

Writing the Discussion Section

The Discussion section is the intellectual core of the report. It should address:

1. **Cross-domain dependencies:** Which design decisions had the most far-reaching downstream consequences? The packet format (Module 1) likely propagated the furthest (through parser, feature vector, dataset, classifier). The UART baud rate (Module 1) likely imposed the tightest constraint on system throughput.
2. **Simulation vs. hardware:** Quantify the sim-to-real gap measured in Module 4. Discuss which features contributed most to the distribution shift and what this implies for simulation fidelity.
3. **Redesign decisions:** If starting over, which single design decision would you change? Candidates include: upgrading to serial peripheral interface (SPI) for receiver-to-FPGA communication (higher throughput), using a wider feature vector (more discriminative features for ML), collecting more training data at low SNR (better classifier robustness).
4. **Scalability:** How would the system change to support 10 beacons? 100? What are the architectural bottlenecks?
5. **Pedagogical reflection:** Which cross-domain skill was hardest to develop? Where did the framework's staged development approach help, and where did it create artificial difficulties?

6.3 Presentation Requirements

Table 12: Presentation requirements.

Parameter	Requirement
Duration	20 minutes presentation + 10 minutes Q&A
Audience	Faculty advisor, program committee, and peers
Format	Slide deck (PowerPoint, Beamer, or Google Slides)
Live Demo	Recommended: 2–3 minute live demonstration of real-time classification

6.3.1 Suggested Slide Outline

1. **Title Slide** (1 slide): Project title, student, advisor, institution.
2. **Motivation & Goals** (2 slides): Why this framework? What cross-domain skills does it teach?
3. **System Architecture** (2 slides): Full system diagram; hardware photo; data flow overview.
4. **Module Highlights** (5 slides, 1 per module): Key design decision, key result, and how it connects to the next module. Use one compelling figure from each module.

5. **Live Demo** (1 slide + live): Show the inference script classifying live beacon transmissions. Change a beacon's SF during the demo and show the classifier tracking the change.
6. **Key Results** (3 slides): System specification table; classification accuracy summary; stress test findings.
7. **Lessons Learned** (2 slides): Cross-domain dependencies; sim-vs.-hardware gap; what would change.
8. **Conclusion & Future Work** (1 slide): Summary and next steps.
9. **Q&A** (1 slide): Contact information and acknowledgments.

Live Demo Preparation

The live demonstration is the most impactful part of the presentation but also the most failure-prone. Prepare for common demo failures:

- **Bring a backup video:** Record a 2-minute screen capture of the inference script running successfully. If the live system fails (USB port not recognized, beacon battery dead, FPGA bitstream corrupted), play the video instead.
- **Pre-test in the presentation room:** RF propagation in the presentation room may differ from the lab. Run the system for 5 minutes in the actual room before the presentation.
- **Minimize configuration changes during demo:** Changing one beacon's SF is impressive; changing multiple parameters simultaneously risks confusion and failure. Keep the demo simple and rehearsed.
- **Have the system running before you reach the demo slide:** Start the beacons and inference script during the system architecture slides so the system is warmed up and stable when you switch to the live view.

Week 4 Deliverable

Final Project Report, Presentation, and Curriculum Package — Consisting of:

1. Final project report (15–20 pages, IEEE format) following the structure in Table 11.
2. Presentation slides (18–20 slides) following the suggested outline.
3. Live demonstration (or backup video) of real-time classification.
4. Complete curriculum package (all 12 items from Table 10), submitted as a structured archive.

7 Complete System Architecture: Final Validated Pipeline

Figure 1 shows the complete system architecture as validated through Module 5 stress testing and benchmarking. All components, interfaces, and data flows have been characterized end-to-end under both nominal and stressed conditions.

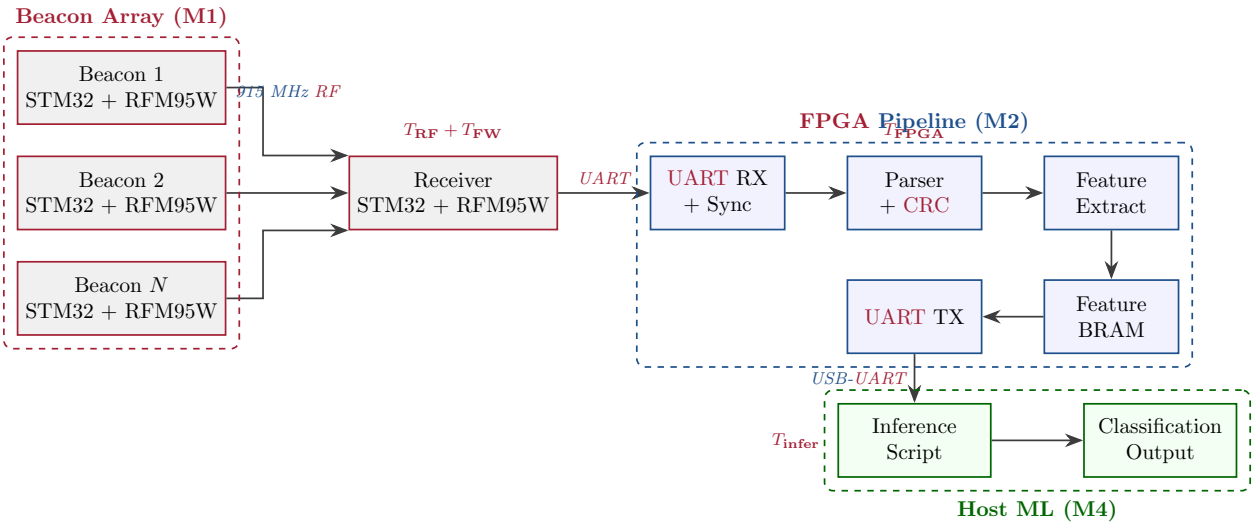

Figure 1: Complete validated system architecture. Module labels (M1–M4) indicate which module developed each subsystem. Module 3 validated the interfaces between subsystems. Module 5 characterized the end-to-end behavior under stress. Timing labels show the latency decomposition from Section 4.

8 Hardware and Software Reference

Table 13: Required hardware and software for Module 5.

Item	Qty / Version	Purpose
STM32 Nucleo-L476RG + RFM95W	≥ 3	2–3 beacons + 1 receiver
915 MHz Whip Antenna	≥ 3	RF transmission/reception
Digilent Nexys A7-100T	1	FPGA processing platform
USB Micro-B Cables	4–5	Power, programming, UART
Breadboard + Jumper Wires	—	Inter-board connections
Oscilloscope or Logic Analyzer	1	Response time measurement (GPIO toggle)
Tape Measure	1	Distance measurement for S3
Physical Obstruction	1	Signal blockage for S3e
AMD Vivado Design Suite	2024.1+	ILA debugging, bitstream verification
STM32CubeIDE	Latest	Beacon/receiver firmware
Python 3.10+	—	Inference script, data analysis
All Module 4 Python dependencies	—	`pandas`, `scikit-learn`, `torch`, etc.
LaTeX Distribution	—	Final report typesetting
Presentation Software	—	Slides (PowerPoint, Beamer, etc.)

9 Assessment Rubric

Table 14: Module 5 assessment rubric.

Assessment Item	Weight	Due
Stress Test Report (S1–S4 results, density limits, SNR degradation, stability)	20%	Week 1
Benchmarking Report (response time decomposition, spec table, baseline comparison)	20%	Week 2
Failure Analysis & Curriculum Package (failure catalog, cross-domain tracing, full package)	25%	Week 3
Final Report & Presentation (15–20 page report, 20-min presentation, live demo)	35%	Week 4

10 Recommended Resources

10.1 System Testing and Validation

1. B. Beizer, *Software Testing Techniques*, 2nd ed., Van Nostrand Reinhold, 1990. (Stress testing methodology, boundary analysis, combinatorial testing.)
2. J. A. Whittaker, *Exploratory Software Testing*, Addison-Wesley, 2009. (Structured approaches to finding system-level failures.)
3. IEEE Std 829-2008, "Standard for Software and System Test Documentation," IEEE, 2008. (Test plan, test report, and test log formats.)

10.2 Technical Writing and Presentation

1. M. Alley, *The Craft of Scientific Presentations*, 2nd ed., Springer, 2013. (Evidence-based presentation design for technical audiences.)
2. J. G. Paradis and M. L. Zimmerman, *The MIT Guide to Science and Engineering Communication*, 2nd ed., MIT Press, 2002. (Technical report structure, audience analysis, visual design.)
3. IEEE, "IEEE Reference Guide," 2024 ed. [Online]. Available: https://ieeeauthorcenter.ieee.org/ (Citation format for the final report.)

10.3 Cross-Domain System Design

1. N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Addison-Wesley, 2010. (Chapter 1: abstraction layers and cross-level design trade-offs.)
2. E. A. Lee and S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2nd ed., MIT Press, 2017. (Cross-domain modeling of embedded systems spanning hardware, firmware, and software.)
3. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed., O'Reilly, 2022. (Chapter 2: end-to-end ML project pipeline—relevant for the final report's system-level perspective.)

10.4 Project-Specific References

1. Semtech, "SX1276/77/78/79 Datasheet," Rev. 7, 2020.
2. STMicroelectronics, "STM32L476xx Reference Manual," RM0351, Rev. 9, 2023.
3. Digilent, "Nexys A7 Reference Manual," Rev. C, 2022.
4. Xilinx/AMD, "Artix-7 FPGAs Data Sheet: DC and AC Switching Characteristics," DS181, 2023.

11 Framework Retrospective

This section provides a closing reflection on the five-module framework as a whole, intended for inclusion in the final report's Discussion section.

11.1 Cross-Module Dependency Summary

The following table summarizes the primary cross-module dependencies that shaped the system's behavior. Each row identifies a design decision, the module where it was made, and the modules where its consequences were felt.

Table 15: Key cross-module design dependencies.

Design Decision	Made In	Felt In	Consequence
Beacon packet format (32 bytes, fixed fields)	M1	M2, M3, M4	Defines parser structure, feature vector, and classification task scope.
UART at 115200 baud	M1	M2, M3, M5	Limits max frame rate to 268 fps; sets FIFO sizing constraint.
CRC-8 polynomial (0x07)	M1/M2	M3	Mismatch between FW and RTL caused first integration failure.
Feature vector format (32 bytes, 8 features)	M2	M4, M5	Determines classifier input dimensionality and task feasibility.
FIFO depth (64 entries)	M2	M3, M5	Constrains burst tolerance; overflow under high beacon density.
Training data SNR range	M3	M4, M5	Classifier accuracy collapses outside training distribution.
Stratified train/test split	M3	M4	Ensures fair evaluation; prevents class imbalance artifacts.
Label leakage prevention	M4	M4, M5	Forces classifier to learn signal properties, not self-reported fields.

11.2 The Framework's Central Lesson

Why Cross-Domain Integration Is the Subject—Not the Obstacle

In a traditional curriculum, students learn RF design, digital logic, embedded firmware, and machine learning as separate subjects. Each course validates student work within its own domain: firmware is tested against a simulated peripheral, VHDL is verified in simulation, classifiers are trained on clean benchmark datasets. The interfaces between domains—the places where firmware decisions constrain hardware behavior, where hardware decisions shape data characteristics, where data characteristics determine ML outcomes—are treated as implementation details rather than learning objectives.

This framework inverts that relationship. The interfaces are the learning objectives. The beacon packet format is not just a firmware deliverable—it is a data contract that propagates through three downstream modules. The FPGA feature vector is not just a digital design exercise—it is the sole information bridge between the physical RF world and the statistical ML world. The training dataset is not just a collection of numbers—it is an empirical sample of a physical system whose distribution is shaped by every hardware and firmware decision that preceded it.

The student who completes this framework has not merely built a system that classifies RF signals. They have experienced—through direct, sometimes painful encounter with integration failures—the reality that engineering systems are defined by their interfaces, not their components.

End of Module 5 Lesson Plan